

# Comprensión de listas o Comprehension Lists

## Colecciones

Una colección es una estructura que como su nombre indica nos permite almacenar varios elementos. Existen diferentes tipos de colecciones, cada una con diferentes características lo que lo hace más indicado para según qué tipo de almacenamiento.

El tipo más sencillo son las **Tuplas** (tuple), un conjunto invariable de elementos, no tienen que ser del mismo tipo.

Veremos también las **Listas** (list) nos permiten guardar también múltiples objetos y de cualquier tipo, con la peculiaridad de que están ordenados. Disponen de todos los mecanismos para trabajar dinámicamente con ellas, añadiendo o quitando elementos, reordenando.

Otro tipo muy importante son los **Diccionarios** (dict), donde tenemos un conjunto de pares que no están ordenados, formados por una clave (key) y un valor (value). La clave tiene que ser única.

También existen los **Conjuntos** (sets), que almacenan valores sin ordenar y que no pueden estar repetidos. Sólo se pueden guardar elementos inmutables (que no cambian) y que de los que se puede obtener un valor **hash** (algo similar a un id único)

Para cada caso escogeremos la estructura que nos facilite más la programación, donde la propia naturaleza de la estructura nos será muy útil.

Por ejemplo para guardar una lista de propiedades, parece más interesante utilizar un diccionario, donde guardaremos de forma natural el nombre de la propiedad y su valor.

Si necesitamos mantener varios elementos sin que se produzca repetición, parece que un conjunto sería lo más indicado.

## Tuplas

El tipo más sencillo de colección son las **Tuplas**, que son un conjunto invariable de elementos, es decir, una vez creadas no podemos modificarlas, ni añadir ni quitar elementos. Los elementos no tienen que ser del mismo tipo, ni tampoco podemos cambiarlos de valor, es decir sus elementos también tienen que ser invariantes, por lo que no pueden ser variables.

¿Qué datos podemos usar que no necesiten cambio? Por ejemplo los días de la semana:

```
>>> dias = ('L', 'M', 'X', 'J', 'V', 'S', 'D')
```

No hay ningún problema en que se repitan elementos dentro de la tupla.

Las definimos poniendo los elementos que la forman entre paréntesis:

```
>>> coordenadas = (1,3,4)
>>> elementos = ('a',7,'hola qué tal')
```

Podemos acceder a un elemento de la tupla con su índice que irá entre 0 y N-1 siendo N el número de elementos de la tupla, indicando la posición del elemento entre corchetes:

```
>>> elementos[0]
'a'
```

Si intentamos acceder a un elemento más allá del último obtendremos una excepción del tipo *IndexError*.

También podemos acceder a los elementos con índices negativos, en cuyo caso se entiende que índice -1 corresponde al último y accederemos hacia atrás:

```
>>> elementos[-1]
'hola qué tal'
>>> elementos[-2]
7
```

Podemos *sumar* tuplas usando el operador "+" y obtendremos una nueva con los elementos de las 2 sumadas.

Si usamos el operador multiplicación "\*" lo que haremos será repetirla N veces.

También podemos acceder a una parte de la tupla usando el índice de elemento inicial y del final separados por ":", obtendremos una lista con los elementos seleccionados:

```
>>> elementos[1:2]
(7,'hola qué tal')
```

También podemos omitir el número inicial o el final si queremos que empiece en el

principio o acabe al final de la tupla:

```
>>> festivos = dias[5:]
>>> laborables = dias[:5]
```

Se suele llamar rodaja (slice) a esta parte que tomamos de una colección mayor.

También podemos usar un tercer entero para indicar cada cuantos elementos queremos iterar.

¿Te recuerda a algo? Sí: la función **range** es un generador de tuplas de números.

## Iterando

Podemos recorrer los elementos de una tupla usando el bucle *for*:

```
valores = (1,5,2,7) # Crea una tupla llamada valores con los 4 números
indicados
for valor in valores: # itera entre los 4 números de la tupla "valores"
recuperando cada vez uno en la variable "valor"
    print(valor) # imprime el valor que se recuperado en "valor" y vuelve a
repetir el bucle hasta que no queden más valores
```

Podemos conocer el tamaño de una tupla usando el método *len*, lo que nos permite recorrer la tupla usando un bucle *while*

```
valores = (1,5,2,7)
contador = 0 # Usaremos "contador" como un índice para ir recuperando el
elemento de la posición "contador" desde la posición 0 (el primero)
while contador < len(valores): # hasta la posición indicada por el tamaño
"len(valores)"
    print(valores[contador]) # Accedemos al elemento de la posición
"contador" y lo imprimimos
    contador += 1 # pasamos índice siguiente y repetimos
```

Podemos saber si un elemento está en una tupla usando la palabra reservada *in*:

```
>>> 4 in valores
False
```

Que nos dará True o False según si el elemento está o no en la tupla.

```
>>> 'X' in festivos
False
```

Podemos usar esta funcionalidad cuando queramos hacer que el usuario seleccione una opción entre varias y e caso de que elija una opción distinta le volveremos a preguntar:

```
print('1 - Opción primera')
print('2 - Opción segunda')
print('3 - Opción tercera')
print('4 - Opción cuarta')
```

```

print('0 - Para salir')
opcion = input('Seleccione entre estas opciones')
while opcion not in ('0','1', '2', '3', '4'):
    print('Opción no válida')
    opcion = input('Seleccione entre estas opciones')
print(f'Seleccionada la opción {opcion}')

```

### Código comprobación de opciones

También podemos saber cuántas veces aparece un elemento con el método *count()*:

```

>>> valores.count(5)
1

```

Python usa internamente las tuplas en muchos casos, por eso podemos utilizarlas también cuando queremos trabajar con varias variables al mismo tiempo:

```

>>> a,b = 1,2 # Para asignar valores a varias al mismo tiempo
>>> a,b = b,a # Para intercambiar sus valores
>>> a
2
>>> b
1

```

Una vez que sabemos que un elemento está en la tupla podemos encontrar su posición utilizando el método **index**.

```

>>> ganadores = ('Brasil','Italia','Brasil')
>>> ganadores.count('Brasil')
2
>>> ganadores.index('Brasil')
0

```

Si algún elemento está más de una vez podemos hacer que la búsqueda se realice a partir de un elemento indicando el índice a partir del que buscar

```

>>> ganadores.index('Brasil',1)
2

```

## Ejemplo de uso de tuplas: BuzzWord

NASA - MSC  
Houston, Texas

MSC MEMORANDUM

10 October 1966

SUBJECT: Buzzword Generator

1. The Buzzword Generator consists of three columns of buzzwords numbered zero to nine.

| Column 1        | Column 2            | Column 3       |
|-----------------|---------------------|----------------|
| 0. integrated   | 0. Management       | 0. Options     |
| 1. total        | 1. organizational   | 1. Flexibility |
| 2. systematized | 2. monitored        | 2. capability  |
| 3. parallel     | 3. reciprocal       | 3. mobility    |
| 4. functional   | 4. digital          | 4. programming |
| 5. responsive   | 5. logistical       | 5. concept     |
| 6. optimal      | 6. transitional     | 6. time-phase  |
| 7. synchronized | 7. incremental      | 7. projection  |
| 8. compatible   | 8. third-generation | 8. hardware    |
| 9. balanced     | 9. policy           | 9. contingency |

2. The procedure is simple. Think of any three digit number at random. Then select the corresponding buzzword from each column. Put them together and WHAM! POW! ZAP! You sound just like you know what you're talking about.
3. Take for instance the number 257. Take word two from column one, word five from column two and word seven from column three. You now have "systematized logistical projection". You don't know what it means but don't worry, neither do "they".
4. Would you prefer "balanced incremental flexibility". Possibly "parallel reciprocal options". Or maybe "integrated transitional contingency". How about "functional third-generation hardware" and "optimal management mobility". Now that ought to do the trick.
5. The important thing is that the Buzzword Generator provides the user with the perfect aid for preparing anything on the subject of spaceflight. Automatically you have one thousand different combinations, all of which will give you that proper ring of decisive, progressive, knowledgeable, authority.

Vamos a hacer ahora un programa que ya usaban en la mismísima NASA allá por 1966. Se trata de crear un generador de palabros técnicos (BuzzWord), con el fin de poder rellenar informes de jerga.

A partir de 3 columnas de 10 términos generaremos un palabra técnico escogiendo aleatoriamente 1 palabra de cada columna.

Vamos a utilizar tuplas para guardar la colección de elementos que no cambian.

También vamos a usar el generador de números aleatorios para elegir una palabra de cada columna usando la función `randint(0, 9)` que nos va a dar un número aleatorio entre 0 y 9, con los extremos incluidos. Para ello tenemos que incluir el módulo *random*, que es lo que hacemos en la línea `"from random import randint"`.

Más adelante veremos con más detalle el uso de módulos, ahora mismo es suficiente con saber que un módulo es un conjunto de código externo que podemos usar desde nuestro programa.

### Código del Generador de Palabras

### Ejemplo de matrices con tuplas: Cálculo del IRPF

Vamos a hacer ahora otro ejemplo que también usa tuplas: un programa que calcula nuestras retenciones IRPF. A partir del dato de nuestro sueldo calcularemos siguiendo la tabla de retenciones, que almacenaremos en nuestro programa en una matriz formada por tuplas de tuplas de valores:

```
>>> tupla1 = (1,2)
>>> tupla2 = (3,4)
>>> matriz_tupla = (tupla1,tupla2)
```

Accederemos a un elemento usando 2 índices:

```
>>> matriz_tupla[1][0]
```

En nuestro ejemplo usaremos esta matriz de tuplas para guardar los distintos tramos del IRPF:

```
tramos = ((12450, 0.19),
          (20200, 0.24),
          (35200, 0.30),
          (60000, 0.37),
          (300000, 0.45),
          (10e20, 0.47))
```

### Código de Cálculo de IRPF

## Listas

Una lista es una colección en la que los elementos que pueden ser de distintos tipos, están ordenados.

Características de las listas:

- No todos los elementos tienen que ser del mismo tipo.
- La lista es dinámica y puede cambiar de tamaño, es decir, podemos añadir y quitar elementos.
- Los elementos están ordenados, en cuanto a su posición en la lista. No es que por defecto se les aplique un orden al insertarlos. Además dado que pueden ser de distintos tipos, no siempre se puede establecer un orden según su valor.

Si has programado en otro lenguaje, las listas te pueden recordar a los arrays, pero en Python son mucho más dinámicas y versátiles.

Para declararla lo haremos de la siguiente forma:

```
mi_lista = [ 10, "hola", "Pepe",25]
```

Accederemos a cada elemento usando su posición, empezando en 0 y terminando en el elemento N-1, siendo N el número de elementos de la lista.

```
print(mi_lista[2])
```

Podemos modificar el elemento que está en una posición dada:

```
mi_lista = [ 10, "hola", "Pepe",25]
```

```
print("inicialmente",mi_lista[2])
```

```
mi_lista[2] = "Juan"
```

```
print("ahora es",mi_lista[2])
```

También podemos usar un índice negativo siendo **-1** el elemento de la última posición y así hasta llegar al primero.

Si intentamos acceder a un elemento más allá de la última o de la primera obtendremos una excepción **IndexError**

Podemos crear una lista vacía con `lista = []`

## SubListas

También podemos recuperar varios elementos de una lista, es lo que se llama una **sublista**. Para ello indicaremos un rango de valores, que son 2 valores numéricos separados por ":", de la siguiente forma **principio:final**, siendo **principio** el índice del primer elemento de la sublista y **final** el índice posterior al último que vamos a recuperar.

```
>>> print(mi_lista[0:2])
[10, 'hola']
```

Si omitimos el primer valor se entenderá que nos referimos al primer valor de la lista

```
>>> print(mi_lista[:2])
[10, 'hola']
```

y si omitimos el segundo que queremos llegar hasta el elemento final

```
>>> print(mi_lista[2:])
['Juan', 25]
```

Si omitimos los dos tendremos la lista entera

```
>>> print(mi_lista[:])
[10, 'hola', 'Juan', 25]
```

También podemos incluir un tercer número en la sublista (separado por otro :) que indicará cada cuantos elementos contamos al elegirlos (lo que sería el "paso"). Por defecto este número es 1, pero podría ser otro.

```
>>> print(mi_lista[::2])
[10, 'Juan']
```

Una utilidad de este tercer parámetro es invertir el orden de nuestra lista usando **-fi**. Por ejemplo:

```
>>> print(mi_lista[::-1])
[25, 'Juan', 'hola', 10]
```

## Operaciones sobre listas

Si nuestra lista está formada por elementos homogéneos (todo números o todo texto) podremos realizar una serie de operaciones sobre ellas:

- **Ordenación:** podemos ordenar una lista con *mi\_lista.sort()*. Una vez ordenada sabremos que el primero o el último serán el mínimo o el máximo valor.
  - Podemos ordenarla en orden inverso con *mi\_lista.sort(reverse=True)* para . Conviene recalcar que la operación se realiza sobre la propia lista, no es que la llamada nos devuelva una lista ordenada.
  - Por defecto la ordenación de cadenas distingue entre mayúsculas y minúsculas,

```
>>> palabras = ['hola', 'ola', 'Hola', 'hOLA']
>>> palabras.sort()
>>> palabras
['Hola', 'hOLA', 'hola', 'ola']
```



- Podemos hacer que no dependa de mayúsculas/minúsculas usando una función de ordenación diferente, lo que indicaremos con el argumento **key**, por ejemplo usando la función *str.lower* (que usará todos los caracteres como minúsculos antes de hacer la comparación)

```
>>> palabras = ['hola', 'ola', 'Hola', 'hOLA']
>>> palabras.sort(key=str.lower)
>>> palabras
['hola', 'Hola', 'hOLA', 'ola']
```

- Para que no dependa de si la primera letra es mayúscula usaremos *str.capitalize* que usa el primer carácter de cada cadena como mayúscula

```
>>> palabras = ['Hola', 'ola', 'hola', 'hOLA']
>>> palabras.sort(key=str.capitalize)
>>> palabras
['Hola', 'hola', 'hOLA', 'ola']
```

- Obtener el máximo aplicando la función **max** *max(mi\_lista)*.
- Obtener el mínimo aplicando la función **min** *min(mi\_lista)*.
- Calcular su longitud con **len** *len(mi\_lista)*.

## Búsquedas

Una funcionalidad que usaremos muchos son las búsquedas, que podremos hacer con la palabra reservada **in**. Obtendremos como resultado un booleano, siendo *True* que se ha encontrado y *False* en caso contrario.

Conviene destacar que se distingue entre mayúsculas y minúsculas.

```
>>> mi_lista = ['Juan', 'Dionisio', 'Pepe', 'Manolo']
>>> 'Juan' in mi_lista
True
```

Una vez que sabemos que el elemento está en la lista podemos preguntar en qué posición con **index**:

```
>>> posicion = mi_lista.index("Juan")
0
```

Si intentamos buscar un elemento que no está se produce una excepción de tipo **ValueError**.

También podemos contar el número de veces que se repite un valor con **count**:

```
>>> mi_lista.count("Juan")
1
>>> mi_lista.count("juan")
0
```

Como vemos podemos usar la función *count* para ver si un elemento está o no en una lista.

## Edición en listas

Hemos visto cómo cambiar, buscar, contar o acceder a elementos de una lista. Pero se pueden hacer más operaciones como las siguientes, a las que llamaremos a partir de nuestra lista original

- **append(elemento)**: Añade un elemento al final de la lista.
- **clear()**: borra todos los elementos de una lista.
- **insert(posicion, valor)**: inserta en *posicion* el valor indicado, desplazando los siguientes.
- **remove(elemento)**: elimina " **elemento** " de la lista. Para esto mismo podemos usar el "arcaísmo" **del**:  
`del lista[4]`
- **copy()**: devuelve una copia de la lista. Podemos usarlo para crear una nueva si no queremos alterar la original.
- **reverse(lista)**: invierte el orden de la lista.
- **pop()**: Devuelve el último elemento de la lista y lo retira de la lista. También podemos usarlo para eliminar un elemento indicando el índice  
`lista.pop(2)`

Esta función nos muestra que podemos usar una lista como soporte para un funcionamiento más complejo como sería una cola o pila, donde vamos eliminando el último elemento.

- **extend(lista2)**: añade a la lista actual el contenido de lista2. Es equivalente a hacer una suma de las dos listas:

```
python lista1.extend(lista2) lista1 = lista1 + lista2
```

\* **sort()**: Ordena la lista. Si tenemos una lista que mezcla distintos tipos, por ejemplo números y letras nos dará una excepción de tipo **TypeError**.

También podemos ordenar una lista con la función **sorted**:

```
python  
sorted(list)
```

## Recorrer una lista: enumerar

Uno de los usos más frecuentes de las listas es recorrer sus elementos. Ya dijimos que los bucles *for* están pensado para recorrer colecciones. Veamos cómo hacerlo con una lista:

```
amigos = ['Juan', 'Dionisio', 'Pepe', 'Manolo']
for amigo in amigos:
    print(f'Hola {amigo}')
```

A este proceso de recorrer una lista se le denomina enumerar o iterar. A veces necesitamos además de iterar el indicar la posición del elemento. Podemos hacerlo con la palabra reservada **enumerate**, obteniendo tanto el elemento como su posición:

```
amigos = ['Juan', 'Dionisio', 'Pepe', 'Manolo']
for posicion, amigo in enumerate(amigos): # Crea una variable "posicion" y
    otra "amigo" donde va poniendo el índice y el valor de los elementos
    print(posicion, amigo)
```

Este ejemplo nos muestra que podemos utilizar varias variables en un mismo bucle *for*, algo que haremos con frecuencia.

## Creación dinámica de listas

En ocasiones necesitamos generar una lista a partir de otra, realizando una operación sobre sus elementos. Podemos hacerlo con lo que ya sabemos, recorriendo nuestra lista y añadiendo los elementos a una nueva lista:

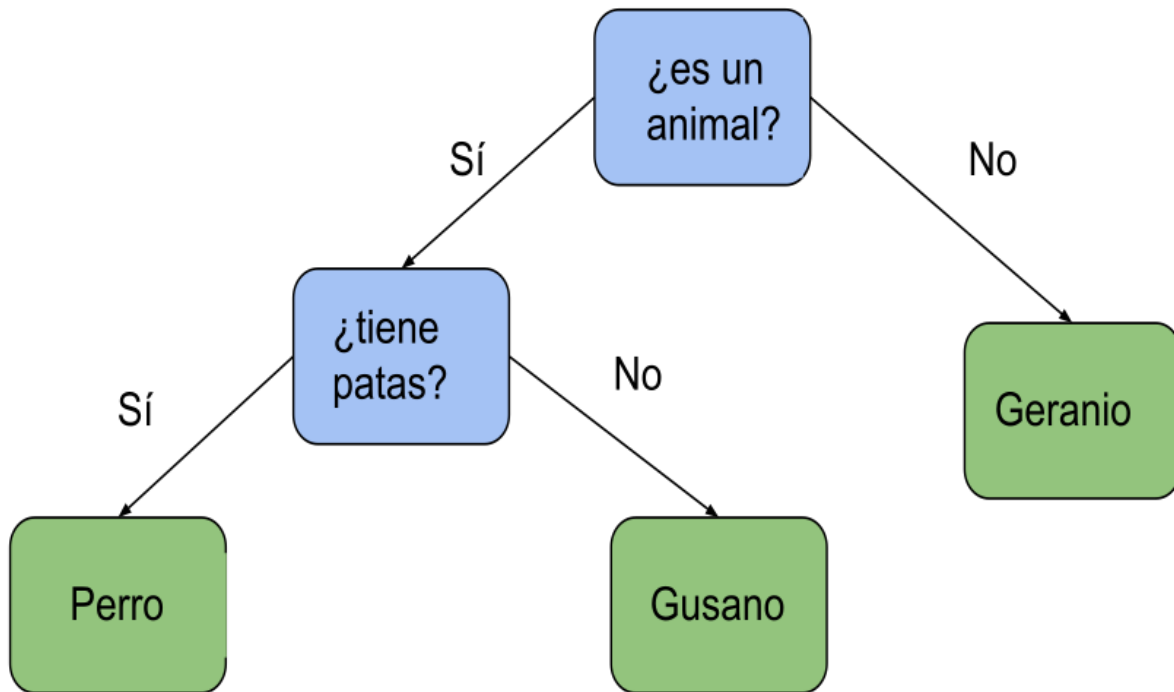
```
pi = 3.14
radios = [2.0, 35, 7.1]
circunferencia = [] # Creamos la lista vacía
for radio in radios:
    circunferencia.append(radio*pi)
```

Pero como ésto es algo muy frecuente, Python incorpora una manera muy cómoda de hacerlo, donde especificamos dentro de la lista la manera en la que calculamos cada elemento a partir de un bucle *for*:

```
pi = 3.14
radios = [2.0, 35, 7.1]
circunferencias = [pi * num for num in radios]
```

## Ejemplo: Juego de las 20 preguntas

Vamos a usar ahora una lista para almacenar una estructura compleja como es un árbol:



Este árbol nos va a permitir navegar entre los nodos que contendrán tanto las preguntas como las respuestas. Cada nodo también guarda referencias a la posición de otros dos:

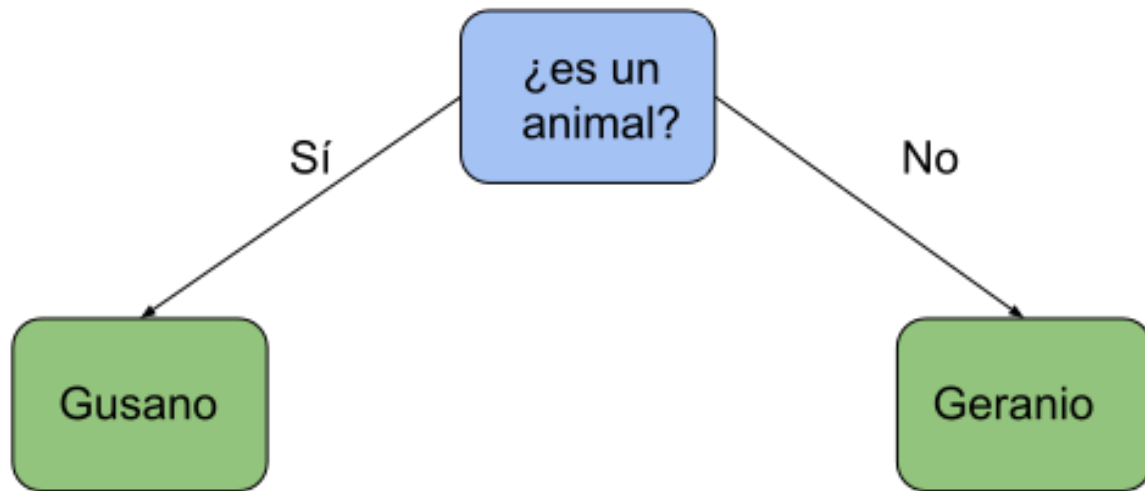
- Al nodo al que ir si la respuesta es *Sí*.
- Nodo al que ir si la respuesta es *No*.

Los nodos que son preguntas tendrán valores válidos para moverse, los llamaremos **nodos rama** y actúan como bifurcación. Los nodos respuesta tienen el valor -1 en los nodos a los que moverse, diremos que estos nodos son **terminales** o **nodo hoja**.

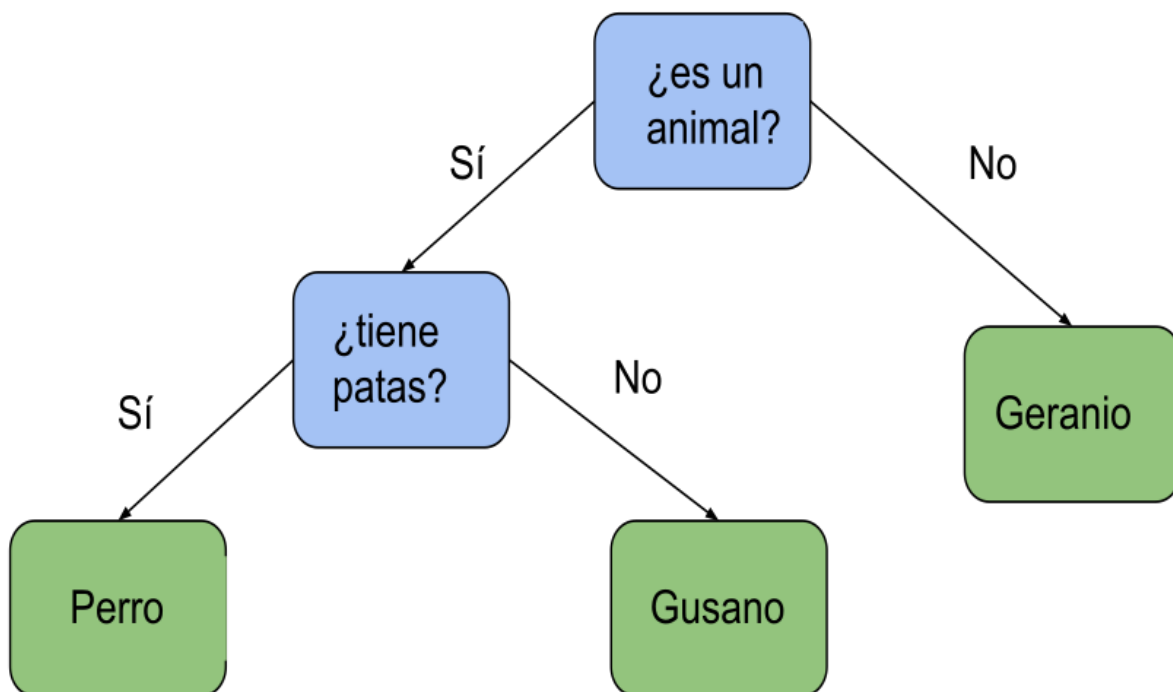
Iremos moviéndonos entre nodos hasta que lleguemos a un nodo terminal que será una respuesta y preguntaremos si es correcta.

Si no lo es, cambiaremos el nodo actual que pasa de ser terminal a ser bifurcación y añadiremos 2 nodos terminales que serán uno la respuesta actual y otro la nueva respuesta. El nodo actual ahora contendrá una nueva pregunta que apuntará a las dos posibles respuestas.

Este es el ejemplo de cambio cuando preguntamos si Gusano es la respuesta y no es el caso.



Este será el árbol después de añadir la nueva pregunta:



Este será el contenido de nuestra lista, donde vemos que los nodos *Respuesta* no conducen a ningún otro y por eso tienen los valores -1, mientras que los nodos *Pregunta* apuntan a otros dos.

| id | text         | Sí | No |
|----|--------------|----|----|
| 0  | es un animal | 2  | 1  |
| 1  | geranio      | -1 | -1 |

|   |             |    |    |
|---|-------------|----|----|
| 2 | tiene patas | 4  | 3  |
| 3 | gusano      | -1 | -1 |
| 4 | perro       | -1 | -1 |

Este programa lo vamos a ir ampliando a lo largo del curso, añadiendo funcionalidad y mejorando la forma en la que almacenamos la información.

[Ejemplo: Juego de las 20 preguntas v1](#)

## Matrices

Hemos dicho que una lista puede contener cualquier cosa. De hecho podemos hacer una lista que contenga listas, es lo que llamamos una matriz. Veamos un ejemplo

```
fila1 = [1, 3, 5]
fila2 = [9, 3, 5]
fila3 = [7, 3, 8]
```

```
matriz = [fila1, fila2, fila3]
```

```
print(matriz) # imprime la matriz
```

```
completa
```

```
print(matriz[1][2]) # imprime un solo elemento el 3º de la fila 2º
```

Como vemos necesitamos 2 índices para acceder a un elemento dado.